



# 모의 코딩테스트 대비 요약본 (Cheat Sheet) - 09.30 업데이트 버전

그동안 누적된 문제 풀이(09.10 이전 내용 포함)와 최근 새롭게 학습하신 알고리즘 패턴(도둑질, 호텔방, 디스크 컨트롤러, 경주로 건설 등)을 하나로 통합한 최신 요약본입니다!

## 1. 파이썬 표준 입출력 (Standard I/O) 템플릿

직접 입출력을 짜야 하는 코딩테스트(백준 등)에서는 시간 초과 방지를 위해 묻지도 따지지도 않고 최상단에 아래 코드를 적습니다.

```
import sys
input = sys.stdin.readline
```

### 1) 자주 쓰는 입력 패턴 모음

```
N = int(input()) # 단일 정수
S = input().rstrip() # 단일 문자열 (rstrip 필수!)
A, B, C = map(int, input().split()) # 띄어쓰기 된 여러 정수
arr = list(map(int, input().split())) # 1차원 리스트
graph = [list(map(int, input().split())) for _ in range(N)] # 2차원 리스트
```

### 2) 무한 입력 (EOF 처리)

```
while True:
    try:
        A, B = map(int, input().split())
        print(A + B)
    except:
        break
```

### 3) 빠른 출력 팁

출력이 많을 때는 한 줄씩 `print()` 하지 말고 모아서 한 번에 출력합니다.

```
arr = [1, 2, 3, 4]
print(*arr) # 1 2 3 4
print('\n'.join(map(str, arr))) # 1부터 4까지 줄바꿈하여 한번에 출력
```

## 2. 자주 사용된 핵심 라이브러리 & 함수 (Python)

### collections (가장 중요!)

- **deque (BFS 필수)**: 양방향 큐. 리스트의 `pop(0)`은  $O(N)$ 이지만, `deque`의 `popleft()`는  $O(1)$ 입니다.
- **Counter**: 리스트 내 원소의 개수를 딕셔너리 형태로 바로 세어줍니다.

```
from collections import deque, Counter

# 1. deque 사용법
q = deque([1, 2, 3])
q.append(4)          # 뒤에 추가
node = q.popleft()   # 앞에서 뽑기 (BFS 핵심)

# 2. Counter 사용법
cnt = Counter(['a', 'b', 'a', 'c'])
print(cnt['a']) # 2
```

### heapq (우선순위 큐, 최단경로)

기본적으로 **\*\*최소 힙(Min Heap)\*\***입니다. (가장 작은 값이 먼저 나옴)

```
import heapq

# 1. 기본 사용법 및 heapify (기존 리스트를 힙으로 변환)
arr = [5, 3, 8]
heapq.heapify(arr)          #  $O(N)$ 으로 한 번에 힙 변환
heapq.heappush(arr, 1)      # 원소 추가
min_val = heapq.heappop(arr) # 1 (가장 작은 값 추출)

# 2. 🔥 상위 K개만 유지하기 (명예의 전당 등)
k = 3
best = []
for score in [10, 50, 20, 40, 30]:
    heapq.heappush(best, score)
    if len(best) > k:
        heapq.heappop(best) # 크기가 k를 넘으면 가장 작은 값 버림
print(best[0]) # 현재 상위 K개 중 최솟값(커트라인) = 30

# 3. 최대 힙 (Max Heap) - 숫자 부호 반전
hq = []
heapq.heappush(hq, -10)
max_val = -heapq.heappop(hq) # 10
```

### itertools & math

백트래킹(완전탐색)과 수학적 계산 시간을 극단적으로 줄여줍니다.

```

from itertools import permutations, combinations
import math

# 1. itertools 사용법
items = ['A', 'B', 'C']
print(list(permutations(items, 2))) # 순열: 순서 다르면 다른 것 [(A,B), (B,A) 등]
print(list(combinations(items, 2))) # 조합: 순서 상관없음 [(A,B), (A,C), (B,C)]

# 2. math 사용법
print(math.gcd(10, 15)) # 최대공약수 = 5
print(math.lcm(10, 15)) # 최소공배수 = 30 (Python 3.9+)
print(math.comb(5, 2)) # 5C2 조합의 '개수' = 10
print(math.isqrt(17)) # 정수 제곱근(내림) = 4
min_cost = math.inf # 무한대 (최솟값 갱신 초기값으로 유용)

```

### 3. bin출 알고리즘 및 접근법 (🔥 심화 패턴 추가)

#### 🔍 1) BFS / DFS (그래프 탐색) 총정리

##### ① 파이썬 재귀 깊이 제한 해제 (DFS 필수)

파이썬은 기본적으로 재귀 깊이가 1000으로 제한되어 있어 깊은 DFS 트리를 타면 `RecursionError`가 발생합니다. 코드 최상단에 무조건 아래 코드를 추가하세요.

```

import sys
sys.setrecursionlimit(10**6)

```

##### ② DFS (깊이 우선 탐색) 템플릿

- **사용처:** 백트래킹(모든 경우의 수 탐색), 맵의 끊어진 영역 개수 세기.
- **재귀(Recursion) 형태:** 가장 짧고 직관적이며 코딩테스트의 90% 이상 쓰입니다.

```

graph = {1: [2, 3], 2: [4], 3: [], 4: []}
visited = set()

def dfs_recursive(node):
    visited.add(node)
    # 필요한 작업 수행
    for next_node in graph[node]:
        if next_node not in visited:
            dfs_recursive(next_node)

```

- **반복문(Stack) 형태:** 트리가 10만 개 이상으로 너무 깊어 재귀 한도를 늘려도 터질 때 사용합니다.

```
def dfs_stack(start_node):
    visited = set()
    stack = [start_node]

    while stack:
        node = stack.pop() # LIFO: 가장 나중에 들어온 것부터 꺼냄 (깊게 파고들기)
        if node not in visited:
            visited.add(node)
            # 스택은 나중에 넣은 것을 먼저 꺼내므로, 번호순으로 가려면 reversed 사용
            for next_node in reversed(graph[node]):
                if next_node not in visited:
                    stack.append(next_node)
```

### ㉓ BFS (너비 우선 탐색) 템플릿

- **사용처:** 최단 거리 구하기 (가중치가 모두 1일 때), 미로 찾기.

```
from collections import deque

def bfs(start_node):
    visited = set([start_node])
    q = deque([start_node])

    while q:
        node = q.popleft() # FIFO: 가장 먼저 들어온 것부터 꺼냄 (넓게 퍼지기)
        for next_node in graph[node]:
            if next_node not in visited:
                visited.add(next_node)
                q.append(next_node)
```

### ㉔ 🔥 [NEW] 방향(코너) 비용이 있는 BFS (경주로 건설)

- 회전할 때 코너 비용(500원)이 드는 등 **\*\*\*어떤 방향으로 진입했느냐\*\*\***에 따라 미래의 최적해가 바뀌는 경우, 2차원 배열 `visited[y][x]`로는 풀 수 없습니다. (비용이 높더라도 꺾지 않고 와서 미래 비용이 싼 루트를 덮어써버림)
- **해결책:** `visited[dir][y][x]` 형태의 3차원 배열을 사용하여 **"진입 방향"별로 최소 비용을 따로 기록해야** 합니다!

### 🧠 2) DP (Dynamic Programming) 총정리

DP는 크게 반복문을 활용하는 **Bottom-Up (타블레이션)**과 재귀를 활용하는 **Top-Down (메모이제이션)** 두 가지 방식으로 구현합니다.

#### ㉓ Bottom-Up 방식 (반복문 템플릿)

- 가장 흔하게 쓰이는 방식으로, 작은 문제부터 차근차근 답을 구해 DP 배열에 채워나갑니다.

```
def dp_bottom_up(n, money):
    # 1. DP 배열 초기화 (초기값 세팅)
    dp = [0] * n
    dp[0] = money[0]
    dp[1] = max(money[0], money[1])

    # 2. 점화식을 통한 반복문 수행
    for i in range(2, n):
        dp[i] = max(dp[i-1], dp[i-2] + money[i])

    return dp[n-1]
```

### ㉔ Top-Down 방식 (재귀 + lru\_cache 꿀팁)

- 파이썬에서는 `functools.lru_cache`를 쓰면 DP 배열을 따로 선언할 필요 없이, 함수 결과값을 알아서 캐싱(저장)해 줍니다! (코딩테스트 꿀팁)

```
import sys
from functools import lru_cache
sys.setrecursionlimit(10**6)

@lru_cache(maxsize=None) # 계산된 재귀 결과를 알아서 메모이제이션 해줌
def dp_top_down(n):
    if n == 0: return money[0]
    if n == 1: return max(money[0], money[1])

    return max(dp_top_down(n-1), dp_top_down(n-2) + money[n])
```

### ㉕ 🔥 [NEW] 원형 배열 DP (도둑질 문제 패턴)

- 첫 집과 마지막 집이 원형으로 연결되어 동시에 고를 수 없는 제약이 있을 때 점화식이 꼬입니다.
- 해결책:** 문제를 두 개의 '직선' 배열로 쪼갭니다.

```
def solution_circular_dp(money):
    if len(money) == 3: return max(money)
    n = len(money)

    # 1. 첫 번째 집을 무조건 터는 경우 (마지막 집 인덱스 무시)
    dp1 = [0] * n
    dp1[0] = dp1[1] = money[0]
    for i in range(2, n - 1): # 마지막 집(n-1) 전까지만
        dp1[i] = max(dp1[i-1], dp1[i-2] + money[i])

    # 2. 첫 번째 집을 절대 안 터는 경우 (마지막 집 포함)
    dp2 = [0] * n
    dp2[0] = 0
    dp2[1] = money[1]
```

```
for i in range(2, n):
    dp2[i] = max(dp2[i-1], dp2[i-2] + money[i])

return max(max(dp1), max(dp2))
```

### 🕒 3) 스케줄링 및 운영체제 로직 (디스크 컨트롤러)

- 🔥 [NEW] 이중 큐 스케줄링:
  - 특정 시점에 들어온 작업 중 가장 짧은 것을 처리해야 할 때(SJF), 큐 2개를 씁니다.
  - waits** (대기 큐): 요청 시간 순 정렬
  - prioritys** (실행 큐): 소요 시간 순 정렬
  - end** (현재 시간) 이전까지 도착한 모든 **waits**를 꺼내 **prioritys**에 쏟아붓고, 가장 짧은 놈을 하나 실행해 **end**를 전진시키는 깔끔한 와일문 로직을 설계합니다.

### 🔗 4) 해시 + Union-Find 개념 (호텔 방 배정)

- 🔥 [NEW] 다음 가볼 곳 매핑하기:
  - k**가 매우 큰데 빈자리를 1씩 더하며 일일이 찾으면  $O(N)$ 으로 터집니다.
  - 딕셔너리로 **rooms[n] = 다음으로 탐색할 번호**를 저장하고, 한 번 탐색해서 찾았다면 지나온 모든 경로의 딕셔너리 값을 도착지로 업데이트(경로 압축)하면  $O(1)$  수준으로 탐색 가능합니다.

### 📅 5) 투 포인터 / 슬라이딩 윈도우

- 연속된 부분 배열의 합/길이 탐색 시  $O(N)$ 에 해결. **start**, **end**를 조작.

### 🎮 6) 격자 시뮬레이션 (공원 산책)

- 🔥 [NEW] 조건부 원상복구:
  - 한 칸씩 반복해서 이동하며 끝까지 문제없을 때만 반영해야 하는 경우, 중간에 장애물을 만나 **break**하면 좌표가 허공에 뜹니다.
  - 이동 전 좌표를 미리 **nx**, **ny**에 임시 저장해두고, 무사히 반복문을 모두 마쳤을 때만 실제 **x**, **y**를 갱신하는 패턴을 써야 합니다.

## 4. 자주 나오는 핵심 구현 (진수 변환, 소수 판별)

### 🗺️ 1) N진수 변환 함수

```
def to_n_base(n, q):
    if n == 0: return '0'
    rev_base = ''
    while n > 0:
        n, mod = divmod(n, q)
        rev_base += str(mod)
    return rev_base[::-1] # 역순인 것을 뒤집어 줘야 함
```

### 🛡️ 2) 소수 판별 (Prime Number)

- 단일 숫자는 제곱근(`math.isqrt(n)`)까지만 확인하면  $O(\sqrt{N})$ 에 해결.

```
import math
def is_prime(x):
    if x < 2: return False
    for i in range(2, math.isqrt(x) + 1):
        if x % i == 0: return False
    return True
```

---